Informe de seguridad del frontend — PharmaSoft (Sesión 11)

Lenguaje de Programación II · Actividad autónoma, Sesión 11 · Autor: Tineo · Repositorio **github.com/Legends600/pharma-frontend**, rama `feature/seguridad-spa-pruebas-tineo` · Backend Practica, rama `feature/seguridad-pruebas-tineo` · Ejecución: 22/10/2026 · Matriz completa: `docs/matriz-seguridad-spa-s11.md`

Se probó la SPA segura del Reto 01 con 20 casos (16 base y 4 propios, de grupos distintos) y luego las dos funcionalidades nuevas de la parte B (aviso previo a la expiración y página «Mi cuenta»), con 9 casos más. Las pruebas se ejecutaron en Chromium automatizado con Playwright. Cada captura indica la URL y los datos leídos del navegador (código HTTP, encabezado `Authorization` recortado y `sessionStorage`); ninguna muestra un token completo.

C1. Resumen de la ejecución

Grupo	Casos	Pasan	Fallan	Hallazgo
Login	5 (L-01 a L-04, L-05*)	**4**	**1**	H-01 (L-03), corregido en `f03ecc8`
Sesión	5 (S-01 a S-04, S-05*)	**4**	**1**	H-02 (S-04)
Interceptor	3 (I-01 a I-03)	**3**	0	—
Guards	3 (G-01, G-02, G-03*)	**3**	0	—
Menú y acciones	2 (M-01, M-02)	**2**	0	—
401 / 403	2 (E-01, E-02*)	**2**	0	—
Aviso de expiración (B1)	5 (B-01 a B-04, B-05*)	**5**	0	—
Mi cuenta (B2)	4 (MC-01 a MC-04)	**4**	0	—
Total	**29**	**27**	**2**	* caso propio

Parte B. El aviso usa `segundosRestantes` (signal que actualiza un `setInterval` de 1 s) y `porVencer` (computed: hay sesión y faltan ≤ 120 s) en `AuthService`; el componente `shared/components/aviso-sesion` se coloca encima de `<router-outlet />` con `role="status"`, muestra mm:ss, permite «Cerrar sesión ahora» y ocultarse con «×». `cerrarSesion()` siempre llama a `clearInterval` (`auth-service.ts:72`), lo que B-05 verificó interceptando la llamada. «Mi cuenta» (`/mi-cuenta`, solo `authGuard`) consume `obtenerPerfil()` → `GET /auth/me`, agrupa los permisos por módulo y compara los del servidor con los de la sesión (`mi-cuenta.ts:40`). El backend devuelve el nombre en `empleadoNombre` (DTO `UsuarioResponseDTO`), no en `nombreCompleto`; el modelo `Perfil` refleja ese contrato real.

C2. Fichas de hallazgos

Hallazgo	**H-01 · Un doble clic en «Ingresar» envía dos POST /auth/login**
Caso / severidad	L-03 · **Baja** (no expone datos, pero duplica la autenticación, abre dos sesiones y deja dos tokens válidos emitidos).
Descripción	Con datos correctos, el doble clic produjo 2 peticiones `POST /api/v1/auth/login`. El botón usa `[disabled]="enviando()"`, pero el segundo clic llega antes de que Angular vuelva a pintar el botón deshabilitado.
Evidencia	`L-03_red.png` (2 POST) · re-prueba `L-03_reprueba.png` (1, 1 y 1 POST en tres repeticiones).
Causa probable	`features/auth/pages/login/login.ts`, método `ingresar()`: confiaba solo en el atributo `disabled` del botón y no revisaba el estado `enviando` antes de enviar.
Corrección	**Aplicada** en `f03ecc8`: `if (this.enviando()) return;` al inicio de `ingresar()` (`login.ts:36`). La regla queda en el código, no solo en la vista.

Hallazgo	**H-02 · Cerrar sesión en una pestaña no la cierra en otra pestaña abierta desde la SPA**
Caso / severidad	S-04 · **Media** (en una computadora compartida, el trabajador cree haber salido y la otra pestaña sigue operando con su token hasta que venza).
Descripción	Una pestaña nueva escrita a mano pide login, porque `sessionStorage` es por pestaña. Pero una pestaña abierta desde la SPA (`window.open`, enlace con `target="_blank"` o «Duplicar pestaña») **copia** el `sessionStorage`. Tras «Cerrar sesión» en la primera, en la segunda `GET /api/v1/ventas` respondió 200: el backend es sin estado y el token sigue vigente.
Evidencia	`S-04_pestana-nueva.png`, `S-04_pestana1-cerrada.png`, `S-04_pestana2-sigue.png`.
Causa probable	`core/auth/auth-service.ts`, método `cerrarSesion()`: solo borra la sesión de la pestaña actual (signal + `sessionStorage`) y no avisa a las demás pestañas.
Corrección propuesta	Difundir el cierre con `BroadcastChannel('pharmasoft.sesion')`: `cerrarSesion()` publica `{ tipo: 'cerrar' }` y cada pestaña, al recibirlo, borra su sesión y va al login. (El evento `storage` no sirve con `sessionStorage`, porque no se comparte entre pestañas.) Como defensa del servidor, una lista de tokens revocados o un `exp` más corto.

C3. Preguntas de análisis

1. **¿Por qué el backend debe seguir validando cada permiso?** El menú (`computed`), la directiva `*appSiPermiso` y `permisoGuard` (`permiso-guard.ts:14`) se ejecutan en el navegador y leen datos que el propio usuario controla: el código JavaScript y `sessionStorage`. En E-01 la vendedora no veía ningún botón «Dar de baja» (0 en pantalla). Aun así, bastó llamar a `darDeBaja()` desde la consola para que saliera un `DELETE /api/v1/productos/58`. Lo único que lo detuvo fue la regla `.hasAuthority(CATALOGO_ESCRIBIR)` de `SecurityConfig.java:87`, que respondió 403. E-02 lo confirma con un `POST /api/v1/ventas` forzado por el almacenero. Ocultar mejora la experiencia; proteger es responsabilidad del servidor, que valida la firma del token y los permisos en cada petición.
2. **sessionStorage, localStorage o cookie HttpOnly.** `sessionStorage` y `localStorage` son legibles desde JavaScript, así que un XSS podría robar el token. `localStorage` además nunca se borra: en la caja compartida, el siguiente trabajador encontraría la sesión abierta. `sessionStorage` sobrevive al F5 (S-01) y se borra al cerrar el navegador. Además la SPA descarta la sesión vencida o dañada (S-02, S-03 en `leerSesionGuardada()`, `auth-service.ts:127`). Su costo es el hallazgo H-02 entre pestañas. Una cookie `HttpOnly; Secure; SameSite=Strict` no es accesible desde JavaScript (resiste XSS), pero exige cambios en el backend (emitir y leer la cookie, protección CSRF y CORS con credenciales). Para la botica hoy elegiría `sessionStorage` con vencimiento corto, aviso previo y la difusión del cierre de H-02. Como evolución, la cookie HttpOnly cuando el backend la soporte.
3. **¿Por qué 401 cierra la sesión y 403 no?** Un 401 significa que el servidor no reconoce al usuario: el token falta, venció o fue alterado (I-03). Seguir con esa sesión solo produciría más 401, así que `errorInterceptor` (`error-interceptor.ts:24`) cierra la sesión y vuelve al login recordando la página. Un 403 significa que el usuario sí está identificado pero esa acción no le corresponde (E-01, E-02): su sesión sigue siendo válida para todo lo demás, como demuestra el `GET /ventas` → 200 posterior. Al revés, un solo botón prohibido expulsaría a la vendedora y perdería la venta a medio registrar. Y un token vencido dejaría la pantalla llena de errores sin llevarla al login. Tampoco debe cerrarse ante un error de conexión (status 0, I-02).
4. **Redirección abierta y L-04.** Una redirección abierta ocurre cuando una aplicación confiable envía al usuario a cualquier URL recibida por parámetro. Un atacante comparte `…/login?returnUrl=https://sitio-falso.com`, la víctima inicia sesión en la página real y termina en un clon que le pide de nuevo la contraseña. `destino()` (`login.ts:60-62`) solo acepta valores que empiezan con `/` y no con `//`. En L-04, `https://sitio-falso.com` no empieza con `/` y en L-05 (propio) `//sitio-falso.com` es una URL relativa al protocolo. En ambos casos el usuario terminó en `/inicio`. Además se navega con `navigateByUrl`, que nunca sale de la SPA.